KANDOR

Stack Tecnologico Completo

Architettura, Servizi e Tecnologie

Marzo 2026

Sviluppato per Studio Serra

Questo documento spiega in dettaglio tutte le tecnologie e i servizi utilizzati nell'app KANDOR, come funzionano e perché sono stati scelti.

Architettura Generale

```
[Browser Utente] ?? [Render (Server)] ?? [Supabase (Database)]
?
[Telegram Bot API]
```

L'utente accede dal browser a Render (che ospita l'app Django). L'app salva/legge i dati su Supabase (database PostgreSQL cloud). Il bot Telegram comunica con l'app tramite webhook.

---

1. ? Django (Framework Web)

Perché Django?

- Admin panel automatico: Django genera automaticamente un pannello di gestione completo per tutti i modelli dati (Dipendent
- ORM integrato: Permette di interagire col database usando Python, senza scrivere SQL diretto.
- Sicurezza: Autenticazione, protezione CSRF, gestione permessi ? tutto built-in.
- Scalabile: Usato da Instagram, Pinterest, Spotify per gestire milioni di utenti.

Componenti Django usati in KANDOR:

- django.contrib.admin ? Il pannello admin personalizzato
- django.contrib.auth ? Sistema login/utenti/permessi
- djangorestframework ? Per eventuali API REST
- django.db.models ? Definizione dei 15 modelli dati
- Template engine ? Rendering delle pagine HTML (Jolly, Riconciliazione, Manuale, ecc.)

---

2. ? PostgreSQL + Supabase (Database)

Perché PostgreSQL?

- Il database più avanzato al mondo per applicazioni web
- Supporto completo per relazioni complesse (foreign key, many-to-many)
- Affidabilità: Usato da aziende come Apple, Netflix, Uber

Perché Supabase?

- Gratuito per progetti piccoli/medi (fino a 500MB di dati)
- PostgreSQL puro: Nessuna limitazione rispetto a un database tradizionale

- Hosting cloud: Il database è sempre accessibile da qualsiasi luogo
- Backup automatici: Supabase gestisce i backup per te
- Dashboard web: Puoi vedere e gestire i dati anche da supabase.com

Come si collega?

L'app Django si collega a Supabase tramite variabili d'ambiente configurate su Render:

- SUPABASE_DB_HOST ? Indirizzo del server
- SUPABASE_DB_NAME ? Nome del database (postgres)
- SUPABASE_DB_USER ? Utente
- SUPABASE_DB_PASSWORD ? Password
- SUPABASE_DB_PORT ? Porta (5432)

---

3. ? Render (Hosting / Deployment)

Come funziona Render?

1. Collegato a GitHub: Ogni volta che fai git push, Render automaticamente:

- Scarica il nuovo codice
- Installa le dipendenze (pip install -r requirements.txt)
- Esegue lo script di build (python build_render.py)
- Riavvia l'app

2. Web Service: L'app gira come servizio web su un server Linux
3. URL pubblico: Accessibile da chiunque su kandor.onrender.com

Configurazione su Render:

- Build Command: pip install -r requirements.txt && python build_render.py
- Start Command: gunicorn cleanup_erp.wsgi
- Variabili d'ambiente: Database, Django secret key, token Telegram

Limitazioni Piano Free:

- L'istanza si addormenta dopo 15 min di inattività (~50s per risvegliarsi)
- Risorse limitate (512MB RAM)
- Upgrade a $7/mese per avere istanza sempre attiva

---

4. ? Gunicorn (Web Server Python)

Perché Gunicorn?

- Django ha un server di sviluppo (manage.py runserver) che non va usato in produzione perché non è sicuro né performante.

- Gunicorn è il server di produzione standard: gestisce connessioni multiple, è robusto e veloce.
- Su Render, il comando di avvio è: gunicorn cleanup_erp.wsgi

---

5. ? WhiteNoise (File Statici)

Perché WhiteNoise?

- Normalmente i file statici (CSS, JS, immagini) vengono serviti da un server web separato (Nginx, Apache)
- WhiteNoise elimina questa necessità: li serve direttamente da Django
- Perfetto per Render dove non hai un server web separato
- Durante il build, collectstatic raccoglie tutti i file statici in una cartella staticfiles/

---

6. ? GitHub (Codice Sorgente)

Come funziona il flusso:

```
[Modifichi il codice] ? [git push su GitHub] ? [Render rileva la modifica] ? [Deploy automatico]
```

Ogni modifica al codice viene:

1. Salvata nel repository GitHub (storico completo)
2. Deployata automaticamente su Render
3. Tracciabile: puoi tornare a una versione precedente in qualsiasi momento

---

7. ? Telegram Bot API

Come funziona?

1. Bot creato tramite @BotFather su Telegram
2. Token salvato in settings.py come TELEGRAM_BOT_TOKEN
3. Webhook configurato su kandor.onrender.com/telegram-webhook/

Flusso di un messaggio:

```
[Dipendente scrive al Bot] ? [Telegram invia al Webhook] ? [Django processa] ? [Risponde al dipendente]
```

- Il webhook è un URL a cui Telegram invia automaticamente ogni messaggio ricevuto dal bot
- Django lo processa in utils_kandor_chat.py e risponde tramite l'API Telegram

---

8. ? Geopy + Nominatim (Geocoding)

Come funziona?

- Quando inserisci un indirizzo per un dipendente o cliente, il sistema chiama Nominatim
- Nominatim restituisce le coordinate GPS
- Le coordinate vengono usate nel Trova Jolly per calcolare la distanza casa-cliente

Perché Geopy + Nominatim?

- Gratuito: Nessun costo (Google Maps API è a pagamento)
- Open source: Nessuna dipendenza da servizi proprietari

9. ? Pandas + XlsxWriter (Excel)

- Pandas: Legge i file Excel importati e li trasforma in dati per il database
- XlsxWriter: Genera i template Excel scaricabili con formattazione

10. ? Certifi (Certificati SSL)

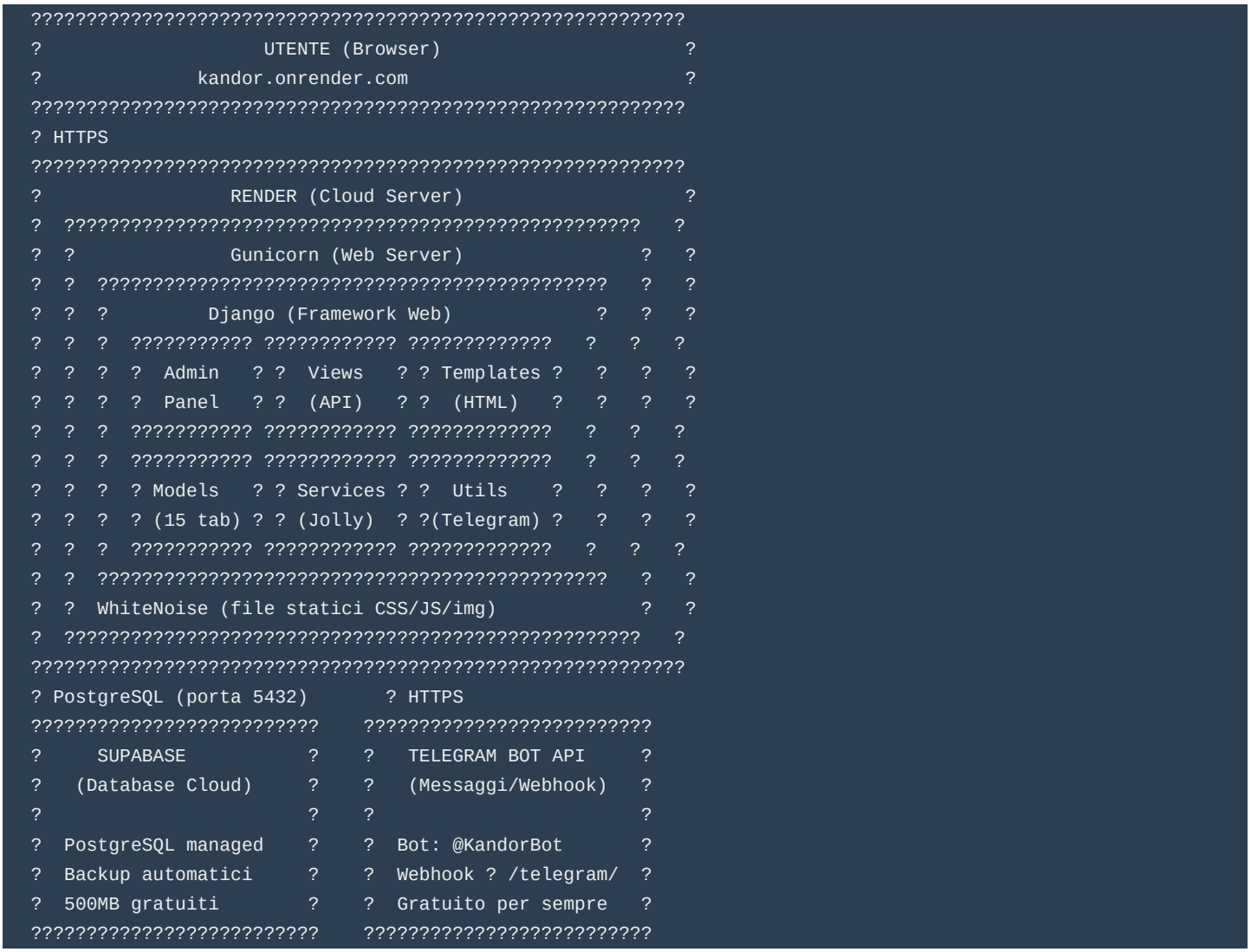
11. ?? Pillow (Immagini)

Riepilogo Costi

Servizio	Piano	Costo
Render	Free	?0/mese
Supabase	Free	?0/mese
GitHub	Free	?0/mese
Telegram Bot API	Free	?0/sempre
Nominatim/Geopy	Free	?0/sempre
Dominio .onrender.com	Incluso	?0/sempre
TOTALE	?0/mese	

> *Per eliminare l'attesa di ~50s e avere prestazioni migliori: Render Starter a $7/mese (~?6.50/mese).*

Diagramma Architetturale



> *KANDOR ? Stack 100% gratuito, professionale e scalabile.*